

CODEX PREPARATION GUIDE FOR IMC PROSPERITY 4

PURPOSE

This document summarizes the highest-value preparations that can be made so Codex can:

- discover stronger strategies faster,
- write cleaner and safer submission code,
- avoid fake progress caused by local overfitting,
- reuse good public ideas without importing stale competition assumptions,
- operate more like a structured research team than a single prompt-response loop.

The focus is this competition specifically: limited visible data, round-specific products, local backtester dependence, hidden submission slices, and strong sensitivity to execution and inventory behavior.

WHAT CODEX NEEDS MOST

Codex improves the most when the environment answers these questions quickly:

1. What assumptions are official, and what is only inferred?
2. What market type is each product in the current round?
3. Which local states actually contain monetizable edge?
4. Is the observed PnL coming from alpha, inventory carry, or local fill assumptions?
5. Which strategy variant is the current frontier, and why?

That means the best preparations are not generic coding helpers. They are competition-specific research infrastructure, validation tooling, and curated knowledge sources.

EXISTING LOCAL BASE

The current repo already provides a useful base:

- Rust backtester and command surface:
 - 'prosperity_rust_backtester/README.md'
 - 'prosperity_rust_backtester/Makefile'
- Local datasets:
 - 'prosperity_rust_backtester/datasets/'
- Core diagnostics:
 - 'scripts/market_discovery.py'
 - 'scripts/forensic_bundle.py'
 - 'scripts/branch_scorecard.py'
 - 'scripts/analyze_artifacts.py'
 - 'scripts/sweep_research_params.py'

- Trader variants and run artifacts:

- 'traders/'
- 'runs/'

This is already enough for normal iteration. The missing pieces are mostly research depth, experiment structure, and reusable external context.

RECOMMENDED RESOURCE STACK

1. CURATED PUBLIC REPO CORPUS

Create a local folder such as:

- 'prosperity_public_corpus/'

Mirror high-quality public Prosperity repositories there, then keep a structured manifest for each repo.

Each repo note should include:

- competition year,
- placement if known,
- repo URL,
- what it is useful for,
- whether it is architecture-only or strategy-relevant,
- transferability notes,
- stale-assumption warnings.

Recommended examples to mirror:

- 'MarkBrezina/Ctrl-Alt-DefeatTheMarket'
- 'ericcccsliu/imc-prosperity-2'
- 'jmerle/imc-prosperity-2'
- recent public Prosperity 4 repos when available

Use public repos for:

- experiment workflow ideas,
- dashboards,
- backtester utilities,
- inventory management patterns,
- logging structure,
- code organization.

Do not use them as authority for:

- current Prosperity 4 product mechanics,
- hidden rules,

- thresholds,
- exact fair values,
- simulator behavior.

2. OFFICIAL ASSUMPTIONS PACK

Maintain one living file such as:

- 'competition_assumptions.md'

Split it into:

- official facts,
- local backtester assumptions,
- inferred hypotheses,
- disproven ideas.

This keeps Codex from repeatedly mixing older Prosperity ideas into current-round reasoning.

3. EXPERIMENT REGISTRY

Create a structured registry:

- 'benchmarks/frontier_registry.csv'
- or 'research.duckdb'
- or 'experiments.sqlite'

Minimum fields:

- trader file,
- run id,
- date,
- dataset and slice,
- total PnL,
- product PnL,
- stress PnL,
- trade count,
- inventory metrics,
- main hypothesis,
- verdict.

This prevents repeated rediscovery of dead ends.

BEST TOOLS TO BUILD

1. SCENARIO LAB

Highest-priority new tool.

Purpose:

- generate hidden-day-like robustness tests,
- make the visible tutorial or round data more informative,
- expose strategies that only work on one lucky slice.

What it should do:

- block-bootstrap from real event sequences,
- preserve local order-book discreteness,
- splice early, mid, and late regimes from different days,
- perturb spreads and displayed volumes,
- test multiple fill assumptions,
- create tail-heavy scenarios where late-day behavior matters more.

Why this matters:

Generic Monte Carlo is too weak for Prosperity because the order book is discrete and regime dependent. A block-bootstrap or regime-splice generator is much more realistic.

2. STATE ALPHA EXPLORER

Purpose:

- automatically score conditional states and rank them by monetizable edge.

Examples of states:

- spread regime,
- ret1,
- rolling deviation,
- top-book imbalance,
- full-depth imbalance,
- microprice delta,
- time-of-day bucket,
- prior move sign,
- run length of last move.

The explorer should report:

- sample count,
- future-mid edge,
- future-fill edge,
- passive vs aggressive monetization,
- per-day consistency,

- submission consistency,
- whether the signal survives stricter execution assumptions.

3. EXECUTION SENSITIVITY HARNESS

Purpose:

- run every serious trader under multiple fill assumptions.

Use the current Rust backtester modes as the starting point:

- default,
- 'trade_match_mode=worse',
- 'queue_penetration=0.5',
- 'trade_match_mode=none'.

Every serious candidate should be scored on:

- default total,
- submission PnL,
- stricter matching total,
- 'none' total,
- inventory shape.

4. COUNTERFACTUAL QUOTE ANALYZER

Purpose:

- answer what would have happened if a quote were one tick closer, one tick farther, or not posted.

This is especially valuable in Prosperity because many strategies lose not from bad alpha, but from quote placement that is too timid or too eager.

5. INVENTORY PATH DASHBOARD

Purpose:

- decompose PnL by inventory state and time.

Track:

- average absolute position,
- time above warning bands,
- PnL while long,
- PnL while short,
- realized vs unrealized path,
- drawdown,

- final inventory dependence.

BEST TOOLS TO INSTALL

DUCKDB

Use for:

- joining run metrics, bundles, CSVs, public-repo metadata, and research tables.

Best use:

- one queryable research database for everything.

POLARS

Use for:

- fast feature engineering and state studies on tick-level data.

Best use:

- replacing ad hoc CSV scripts with faster, cleaner research pipelines.

OPTUNA

Use for:

- constrained parameter search.

Only use it with hard gates, for example:

- minimum training PnL,
- minimum stress PnL,
- inventory cap constraints,
- max drawdown limits.

Do not use it as blind submission-score maximization.

STREAMLIT OR PLOTLY

Use for:

- interactive dashboards over runs and state studies.

Best use:

- browsing fills, positions, and state-conditioned results without rewriting

analysis each time.

MLFLOW OR LIGHTWEIGHT RUN TRACKING

Use for:

- structured experiment comparison once the run count becomes large.

Lower priority than DuckDB, but useful once many strategy branches exist.

AGENT STRUCTURE THAT HELPS CODEX MOST

Codex benefits from explicit role separation.

Recommended roles:

COMPETITION RESEARCH AGENT

Responsibilities:

- official documentation,
- rule changes,
- public-repo discovery,
- current competition context.

Output:

- updated assumptions doc,
- repo shortlist,
- warnings about stale ideas.

ALPHA MINER AGENT

Responsibilities:

- local dataset analysis,
- signal discovery,
- state-conditioned edge studies.

Output:

- ranked signal candidates,
- passive vs aggressive monetization map,
- slice stability report.

EXECUTION AND RISK AGENT

Responsibilities:

- fill sensitivity,
- queue assumptions,
- inventory analysis,
- drawdown decomposition,
- counterfactual quote studies.

Output:

- real edge vs fill-model-assisted edge report.

STRATEGY ENGINEER AGENT

Responsibilities:

- write only the chosen change,
- preserve submission compatibility,
- keep changes narrow and attributable.

Output:

- one clear trader variant,
- implementation notes,
- known risks.

VALIDATOR AGENT

Responsibilities:

- rerun benchmarks,
- compare to controls,
- summarize score, robustness, and inventory metrics.

Output:

- ship or reject recommendation.

REPO LIBRARIAN AGENT

Responsibilities:

- scrape and index public repos,
- extract reusable ideas,
- keep a pattern library.

Output:

- searchable architecture and tooling notes.

PROMPT AND WORKFLOW PREPARATION

Create a small prompt library for Codex.

Examples:

- 'alpha_discovery_prompt.md'
- 'inventory_diagnosis_prompt.md'
- 'execution_falsification_prompt.md'
- 'public_repo_extraction_prompt.md'
- 'strategy_comparison_prompt.md'

Each prompt should force:

- official vs inferred separation,
- product classification,
- one dominant failure mode,
- one narrow change,
- one validation plan.

This reduces prompt drift and keeps strategy work disciplined.

RESEARCH DATABASE STRUCTURE

Use one DuckDB file such as:

- 'research/research.duckdb'

Suggested tables:

- 'ticks'
- 'market_trades'
- 'own_trades'
- 'bundles'
- 'run_metrics'
- 'state_features'
- 'public_repo_manifest'
- 'idea_registry'

High-value derived tables:

- 'state_alpha_summary'
- 'execution_sensitivity_summary'
- 'inventory_path_summary'
- 'frontier_summary'

This makes Codex much better because it can query evidence directly instead of re-

parsing logs every turn.

PUBLIC REPO SCRAPING PLAN

Build a small scraper/indexer that:

- clones selected repos,
- records directory tree,
- extracts README sections,
- indexes files mentioning:
 - inventory,
 - liquidation,
 - dashboard,
 - backtester,
 - risk,
 - spread,
 - market making,
 - pair trading,
 - conversion,
 - volatility,
- tags each repo by relevance.

This should populate:

- 'prosperity_public_corpus/manifest.yaml'
- or a DuckDB table.

CODE QUALITY PREPARATION

To help Codex write better strategy code specifically:

- keep one current frontier file and one benchmark table,
- keep helper functions for:
 - book parsing,
 - fair-value computation,
 - inventory budgeting,
 - passive quote placement,
 - aggressive take logic,
 - traderData serialization,
- enforce a common strategy structure:
 1. read state,
 2. compute features,
 3. compute fair,
 4. aggressive takes,
 5. temporary position update,
 6. passive quotes,
 7. serialize state.

This makes future trader diffs smaller and easier to evaluate.

COMPETITION-SPECIFIC GUARDRAILS

Codex should always be set up to remember:

- older Prosperity repos are inspiration, not truth,
- local backtester assumptions can create fake winners,
- submission-only optimization is dangerous,
- later rounds may have completely different product types,
- strategy complexity should follow diagnosed need, not style.

Every evaluation stack should therefore include:

- full round bundle,
- per-slice view,
- product-level PnL,
- inventory metrics,
- stress modes,
- a note on likely fill-model dependence.

HIGHEST-ROI BUILD ORDER

If only a few preparations are possible, do them in this order:

1. Build 'scenario_lab'.
2. Build 'state_alpha_explorer'.
3. Create a DuckDB research database.
4. Add a dashboard on top of it.
5. Mirror a curated public Prosperity repo corpus.
6. Add a structured experiment registry.
7. Formalize the agent split.

MINIMAL PRACTICAL SETUP

If the goal is the smallest setup that still helps a lot:

- keep the current Rust backtester,
- add DuckDB,
- add Polars,
- add one scenario tool,
- add one state explorer,
- mirror a few strong public repos,
- maintain a frontier registry,
- use at least three Codex roles:
 - research,
 - strategy engineering,
 - validation.

SUMMARY

The best way to help Codex produce stronger Prosperity strategies is to turn the environment into a research system rather than a folder of trader files.

The most valuable additions are:

- a curated public-repo corpus,
- a scenario generator,
- a state alpha explorer,
- execution-sensitivity tooling,
- an experiment registry,
- a DuckDB-backed research layer,
- separate Codex roles for research, engineering, and validation.

That setup helps Codex do better work for this competition because it improves evidence quality, reduces repeated dead ends, and keeps strategy development aligned with the actual Prosperity workflow: hidden slices, round-specific mechanics, and strong sensitivity to execution and inventory.